

Beadandó feladat

Felhőszolgáltatások és CI/CD

AZ-900 témakör: Azure Architecture & Services

CI/CD témakör: GitHub Actions – workflow és GitHub Pages deploy

I. Azure Architecture & Services

Bevezetés

A Microsoft Azure a Microsoft nyilvános felhőplatformja, amely világszerte több mint száz adatközpontból nyújt számítási, hálózati, tárolási, adatbázis-kezelési, mesterséges intelligencia és integrációs szolgáltatásokat. Az AZ-900 (Microsoft Azure Fundamentals) vizsga egyik központi témaköre az Azure architektúrája és szolgáltatásai, amely megérti a tanulóval, hogyan épül fel a platform fizikai és logikai szinten, hogyan szervezhetők az erőforrások, és milyen főbb szolgáltatáskategóriák állnak rendelkezésre. Az alábbi dokumentum ezt a témakört dolgozza fel részletesen.

Az Azure architektúrájának kétféle nézete

Az Azure architektúrája két nagyobb nézetből vizsgálható. A fizikai (globális) infrastruktúra az adatközpontokat, ezek földrajzi csoportosítását, valamint a hibatűrési zónákat foglalja magába. A logikai (erőforrás-szervezési) hierarchia ezzel szemben azt írja le, hogy a felhasználó által létrehozott erőforrások hogyan helyezkednek el az előfizetési, csoportosítási és kezelési szinteken. A két nézet együttesen biztosítja, hogy az Azure egyszerre legyen földrajzilag elosztott és központilag irányítható.

Globális infrastruktúra: régiók, zónák és régiópárok

A globális infrastruktúra alapegysége az adatközpont, amelyből az Azure régiókba szervez. Egy régió földrajzilag közeli adatközpontok halmaza alacsony hálózati késleltetéssel összekötve. A régiókon belül egy vagy több rendelkezésre állási zóna (Availability Zone) található, amelyek fizikailag különálló, önálló áramellátással, hűtéssel és hálózattal rendelkező adatközpontok. Az Azure régiókat ezen felül régiópárokba szervezi: ezek azonos földrajzi területen lévő, de legalább pár száz kilométerre elhelyezkedő régiók, amelyek között a Microsoft a vész-helyreállítási replikációt és a frissítési ütemezést koordinálja, így például egy regionális kimaradás esetén az adatok a párrégióból visszaállíthatók. A peremhálózati pontok (edge locations) a végfelhasználókhoz közeli kiszolgálók, amelyek főként a tartalomszolgáltatás (CDN, Azure Front Door) gyorsítását szolgálják.

Fogalom	Jelentés
Region	Földrajzilag közeli adatközpontok csoportja; a legtöbb erőforrás létrehozásakor régiót kell választani.
Availability Zone	Régióon belüli, fizikailag elkülönített adatközpont. Zónaredundáns szolgáltatások (pl. ZRS Storage, Zone-Redundant VM) több zónán átívelő rendelkezésre állást biztosítanak.

Fogalom	Jelentés
Region Pair	Két, azonos földrajzi területen lévő, de egymástól távoli régió. Az Azure ezek között végzi a vész-helyreállítási replikációt (GRS/GZRS) és az ütemezett frissítéseket.
Sovereign region	Speciális, szuverén régiók (pl. Azure Government, Azure China) a szabályozási és nemzetbiztonsági követelmények kielégítésére.

Erőforrás-hierarchia: management group, subscription, resource group, resource

A logikai oldalon az Azure négy szintű hierarchiát kínál az erőforrások szervezésére. A legelső szinten maguk az erőforrások (resources) állnak: ilyen például egy virtuális gép, egy tárfiók vagy egy adatbázis. Az erőforrásokat erőforráscsoportokba (Resource Group) gyűjtjük, amelyek az életciklus, a hozzáférés-vezérlés és a költségelszámolás alapegységei. Az erőforráscsoportok mindig egy előfizetéshez (Subscription) tartoznak, amely a számlázási és kvóta-határvonalat jelenti. Végül az előfizetések felett kezelési csoportok (Management Groups) helyezkednek el, amelyekben több előfizetésre egyszerre lehet házirendet (Azure Policy) vagy szerepkört (RBAC) alkalmazni. Ez a hierarchia teszi lehetővé, hogy egy nagyvállalat egyetlen szabályrendszerrel kezelje akár több száz előfizetést is.

Főbb szolgáltatáskategóriák

Az Azure szolgáltatási katalógusa több mint kétszáz tételt tartalmaz, ezek azonban néhány nagyobb kategóriába rendezhetők. Az alábbiakban a legfontosabb csoportokat ismertetjük, az AZ-900 vizsgán is hangsúlyos szolgáltatásokra fókuszálva.

Számítási szolgáltatások (Compute).

- **Azure Virtual Machines (VM):** IaaS modell, teljes operációs rendszer szintű kontroll. Akkor választjuk, ha hagyományos szerveralkalmazást vagy egyedi konfigurációt kell futtatnunk.
- **Azure App Service:** PaaS webalkalmazás-gazdagép. Az OS-szintű karbantartást a Microsoft végzi, a fejlesztőnek csak a kódot kell üzemeltetnie.
- **Azure Functions:** Szerver nélküli (serverless) számítás. Az eseményre induló kódot a platform automatikusan skálázza, és csak a tényleges futási időt számlázza.
- **Azure Kubernetes Service (AKS):** Menedzselt Kubernetes-fürt, amely a konténerizált mikroszolgáltatásokhoz biztosít skálázható futtatókörnyezetet.
- **Azure Container Instances (ACI):** Egyszerű, gyors konténerindítás Kubernetes-fürt nélkül, főként rövid életű feladatokhoz.

Hálózati szolgáltatások (Networking).

- **Virtual Network (VNet):** Az Azure logikailag elszigetelt hálózati alapegysége; alhálózatokra (subnet) bontható, és kapcsolható más VNet-ekhez peering útján.
- **Load Balancer és Application Gateway:** L4, illetve L7 szintű terheléelosztók. Az Application Gateway emellett Web Application Firewall (WAF) szolgáltatást is nyújt.
- **VPN Gateway és ExpressRoute:** Hibrid kapcsolatok az on-premises hálózatokhoz: az előbbi az interneten keresztül, az utóbbi dedikált, privát összeköttetésen.
- **Azure DNS és Azure Front Door:** Névfeloldási, illetve globális, peremhálózati terheléelosztási és gyorsítótárazási szolgáltatások.

Tárolási szolgáltatások (Storage).

- **Blob Storage:** Strukturálatlan adatok (képek, videók, biztonsági mentések, log fájlok) tárolása. Három réteget kínál: Hot, Cool és Archive, amelyek között a hozzáférés gyakoriságának megfelelően választunk.
- **Azure Files:** Felhőben futó SMB és NFS megosztások, amelyek hagyományos hálózati meghajtóként is csatlakoztathatók.
- **Disk Storage:** Virtuális gépek menedzselt lemezei (Standard HDD, SSD, Premium SSD, Ultra Disk teljesítményszinteken).
- **Queue és Table Storage:** Egyszerű üzenetsorok, illetve NoSQL kulcs-érték tárolás kis méretű, strukturált adatokhoz.

A tárolási szolgáltatások redundanciaszintjei (LRS, ZRS, GRS, GZRS) közvetlenül a globális infrastruktúrához kapcsolódnak. Az LRS (Locally Redundant Storage) három másolatot tart egyetlen adatközpontban; a ZRS három másolatot helyez különböző rendelkezésre állási zónákba; a GRS a régiópár távoli régiójába is replikál, a GZRS pedig a zónaredundanciát és a régiók közötti replikációt kombinálja. Ennek a választásnak közvetlen ár- és SLA-vonzata van.

Adatbázis-szolgáltatások (Databases).

- **Azure SQL Database:** Menedzselt relációs adatbázis, Microsoft SQL Server motorral, automatikus mentésekkel, beépített magas rendelkezésre állással.
- **Azure Database for PostgreSQL / MySQL:** Nyílt forráskódú motorokra épülő, szintén menedzselt szolgáltatások.
- **Azure Cosmos DB:** Globálisan elosztott NoSQL adatbázis, többféle API-val (Core SQL, MongoDB, Cassandra, Gremlin, Table) és garantált, milliszekundumos válaszütemmel.

SLA és rendelkezésre állás

Az Azure minden szolgáltatáshoz közzéteszi a szolgáltatási szint megállapodását (SLA), amely az adott szolgáltatás havi rendelkezésre állását rögzíti. Az SLA értéke jellemzően a kiválasztott konfigurációtól függ: egyetlen virtuális gép például csak Premium SSD lemezek mellett kap 99,9%-os SLA-t, míg ugyanaz a VM rendelkezésre állási zónákba telepítve már 99,99%-os SLA-t élvez. Egy összetett alkalmazás eredő SLA-ja a komponens-SLA-k szorzata, ezért egy adott architektúra tervezésekor mindig figyelembe kell venni, hogy a leggyengébb láncszem határozza meg a végeredményt.

Összefoglalás

Összefoglalva: az Azure architektúrájának ismerete azt jelenti, hogy egyszerre értjük a globális infrastruktúrát (régiók, zónák, régiópárok), a logikai szervezést (management group, subscription, resource group, resource), valamint a főbb szolgáltatáscsoportokat (compute, networking, storage, databases). Ezek együttese alkotja azt a fogalmi keretet, amelyre a magasabb szintű Azure-tanúsítások (AZ-104, AZ-204, AZ-305) építenek, és amely a mindennapi felhőtervezési döntések alapját adja.

II. GitHub Actions – workflow és GitHub Pages deploy

Bevezetés: CI/CD a gyakorlatban

A folyamatos integráció (Continuous Integration, CI) és a folyamatos szállítás/telepítés (Continuous Delivery / Deployment, CD) ma a szoftverfejlesztés alapelvárása. A CI célja, hogy a fejlesztők gyakran, kis lépésekben integrálják a kódjukat a közös ágba, és minden integráció automatikus fordítást, tesztelést és kódminőség-ellenőrzést indítson el. A CD ezen továbblépve gondoskodik arról, hogy a sikeresen átvizsgált kód automatikusan eljusson valamilyen célkörnyezetbe – például egy tesztkiszolgálóra, egy konténerregisztrációba, vagy mint ebben a példában, egy statikus weboldal-publikációs felületre.

Mi a GitHub Actions?

A GitHub Actions a GitHub natív CI/CD motorja, amely közvetlenül a repository mellett, YAML formátumú workflow-fájlokkal írható le. Külön server, dedikált build-gép vagy harmadik fél által üzemeltetett CI-szolgáltatás nem szükséges – a futtatást a GitHub által biztosított, illetve saját üzemeltetésű (self-hosted) runnerek végzik. A workflow-k a `.github/workflows/` könyvtárban helyezkednek el, és minden commit, pull request vagy ütemezés alkalmával lefuthatnak.

Alapfogalmak

- **Workflow:** egy YAML-fájllal definiált, eseményvezérelt automatizmus. Egy repositoryban tetszőleges számú workflow lehet.
- **Event (trigger):** az az esemény, amely elindítja a workflow-t (pl. push, pull_request, schedule, workflow_dispatch a manuális indításhoz).
- **Job:** a workflow egy izolált futási egysége, amelyet egy runneren futtat a rendszer. Egy workflow több jobot is tartalmazhat, ezek alapértelmezetten párhuzamosan futnak, de a needs kulcsszóval sorba is rendezhetők.
- **Step:** a jobon belüli egyes lépések. Egy step vagy egy shell parancsot futtat (run: ...), vagy egy újrafelhasználható Action-t használ (uses: ...@verzió).
- **Action:** újrafelhasználható, verzióval ellátott építőelem (pl. actions/checkout, actions/setup-node), amelyet a GitHub Marketplace-en is publikálni lehet.
- **Runner:** az a gép, amelyen a job lefut. A GitHub által hostolt runnerek (ubuntu-latest, windows-latest, macos-latest) ingyenes percekkel érkeznek; nagyobb terhelésnél self-hosted runnert is lehet csatlakoztatni.
- **Secret:** titkosított környezeti változó, amelyet a repository, a környezet vagy a szervezet szintjén lehet megadni, és a workflow-ban a secrets.NAME szintaxissal érhető el.

GitHub Pages mint deploy célpont

A GitHub Pages a GitHub beépített, ingyenes statikus weboldal-tárhelye. Eredetileg egy dedikált gh-pages ág tartalmát publikálta, ma azonban a hivatalosan ajánlott megoldás a GitHub Actions-alapú telepítés: a workflow egy artifactot tölt fel az Actions-hez, majd a deploy-pages action ezt teszi közzé. Ennek előnye, hogy a build és a publikálás teljesen ugyanabban a folyamatban történik, a forrásból semmi nem kerül be a végeredménybe, és a Pages saját, izolált environmentet kap a védelmi szabályok alkalmazásához.

Működő példa: statikus oldal telepítése

Konkrét példa. Az alábbi workflow egy klasszikus statikus weboldalt (HTML/CSS/JS a docs/ könyvtárban) telepít GitHub Pages-re minden main ágra történő push esetén. A teljes fájl a `.github/workflows/deploy.yml` néven kerül a repositoryba.

```
name: Deploy static site to GitHub Pages

on:
  push:
    branches: [ main ]
    workflow_dispatch:           # manuális indítás engedélyezése

# A Pages deployhoz szükséges minimum jogosultságok
permissions:
  contents: read
  pages: write
  id-token: write               # OIDC tokenhez a deploy-pages-nek

# Egyszerre csak egy deploy fusson; az újabb push megszakítja a régit
concurrency:
  group: "pages"
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v5

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: ./docs           # ezt a könyvtárat tesszük közzé

  deploy:
    needs: build                # csak sikeres build után fusson
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

A workflow lépésről lépésre

- **on: push és workflow_dispatch** — kettős trigger. Minden main-re érkező commit indít egy futást, de manuálisan, az Actions felületről is futtatható.
- **permissions** — a legkisebb jogosultság elve. A pages: write a Pages API-hoz, az id-token: write az OIDC-tokenhez kell, amellyel a deploy-pages igazolja, hogy valóban a megfelelő repositoryból és ágból érkezik a publikáció.
- **concurrency** — biztosítja, hogy a Pages környezetbe egyszerre csak egy deploy menjen; a cancel-in-progress: false miatt az éppen futó deploy nem szakad meg, a következő pedig sorban várakozik.
- **build job** — kicheckoutolja a repositoryt, beállítja a Pages environmentet, majd a megadott könyvtárat artifactba csomagolja (a Pages elvárása szerint gzipelt tar formátumban).
- **deploy job** — kizárólag akkor fut, ha a build sikeresen lezárult (needs: build). Az environment blokk teszi lehetővé, hogy a Pages URL automatikusan megjelenjen a futtatás összegzésében; az id: deployment kulcs pedig a steps.deployment.outputs.page_url hivatkozást teszi használhatóvá.

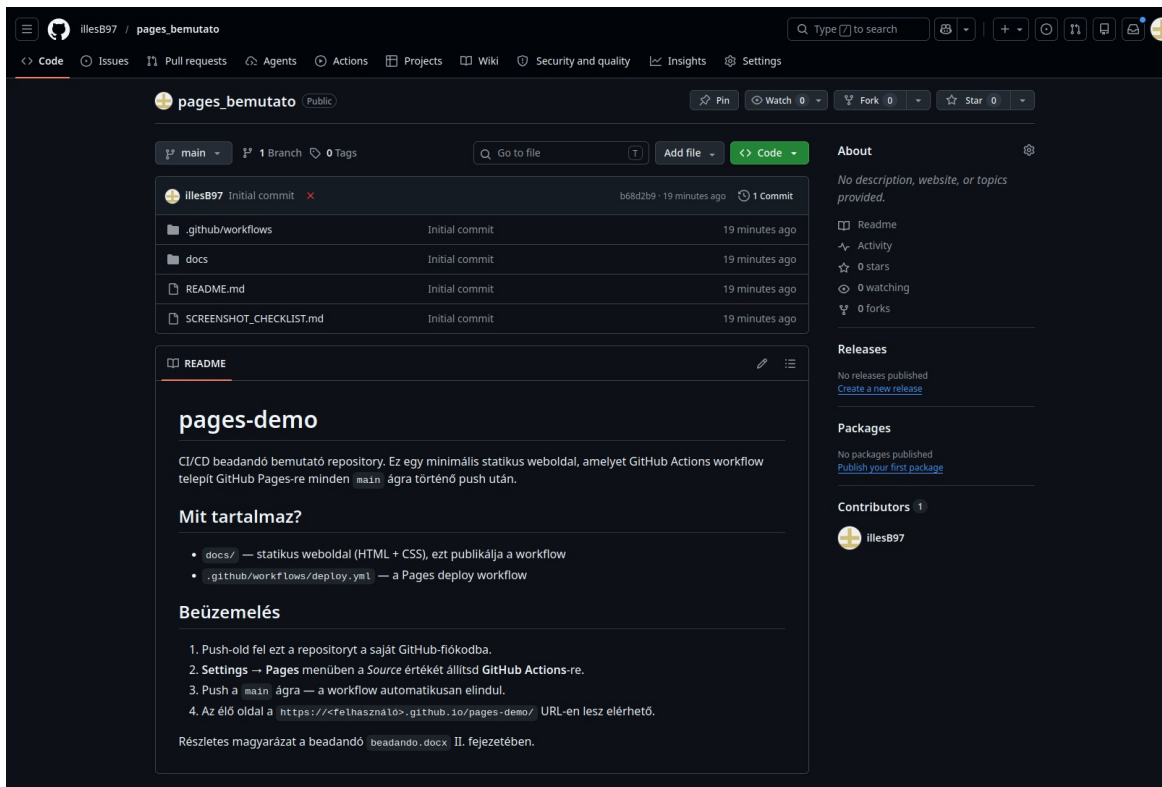
Build-lépés komplexebb projektekhez. Ha az oldal statikus generátorral (pl. Hugo, Jekyll, MkDocs, Vite) készül, a build job kibővíthető egy fordítási lépéssel. Például Node.js alapú projekt esetén:

```
- name: Setup Node
  uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: 'npm'
- name: Install & build
  run: |
    npm ci
    npm run build
- name: Upload artifact
  uses: actions/upload-pages-artifact@v3
  with:
    path: ./dist
```

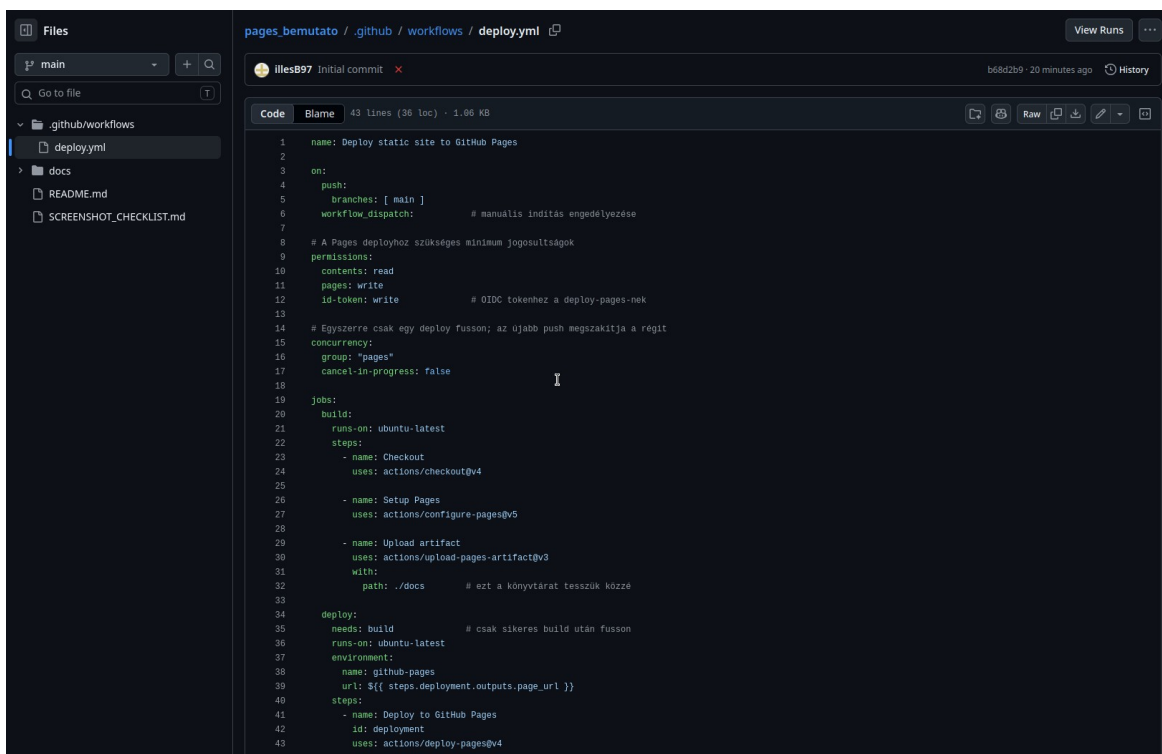
Repository beállítások. A workflow csak akkor fog működni, ha a repository Settings → Pages menüpontjában a Source értékét GitHub Actions-re állítjuk. Ezután a deploy job kimenete a <https://<felhasználó>.github.io/<repo>/> URL-en lesz elérhető, és az egyes futtatások az Actions fülön visszanezhetők, beleértve a build és a deploy lépés naplóját is.

Bemutató screenshotokkal

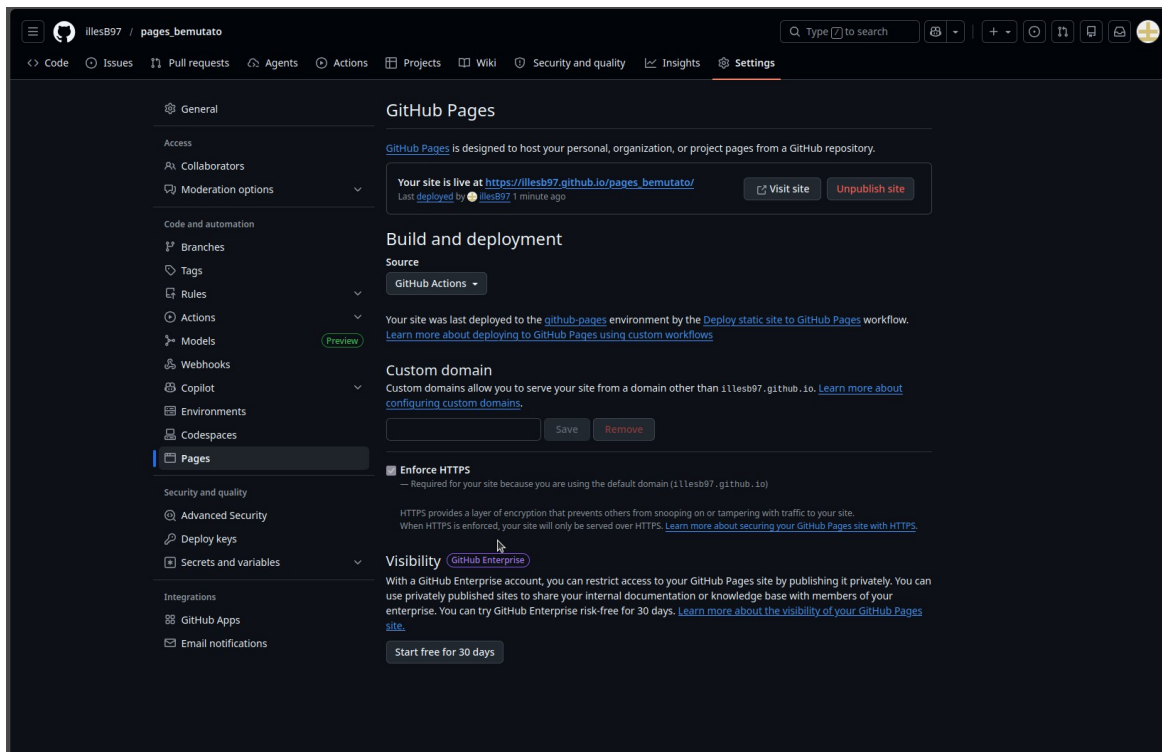
A fenti workflow-t egy demonstrációs célra létrehozott pages_bemutato GitHub-repositoryba feltöltve és a Pages Source értékét GitHub Actions-re állítva az alábbi screenshotok dokumentálják a működést. A képek a folyamat sorrendjében követik egymást: a forráskódtól a Pages konfiguráción és a sikeres workflow-futáson át a publikált, élő weboldalig.



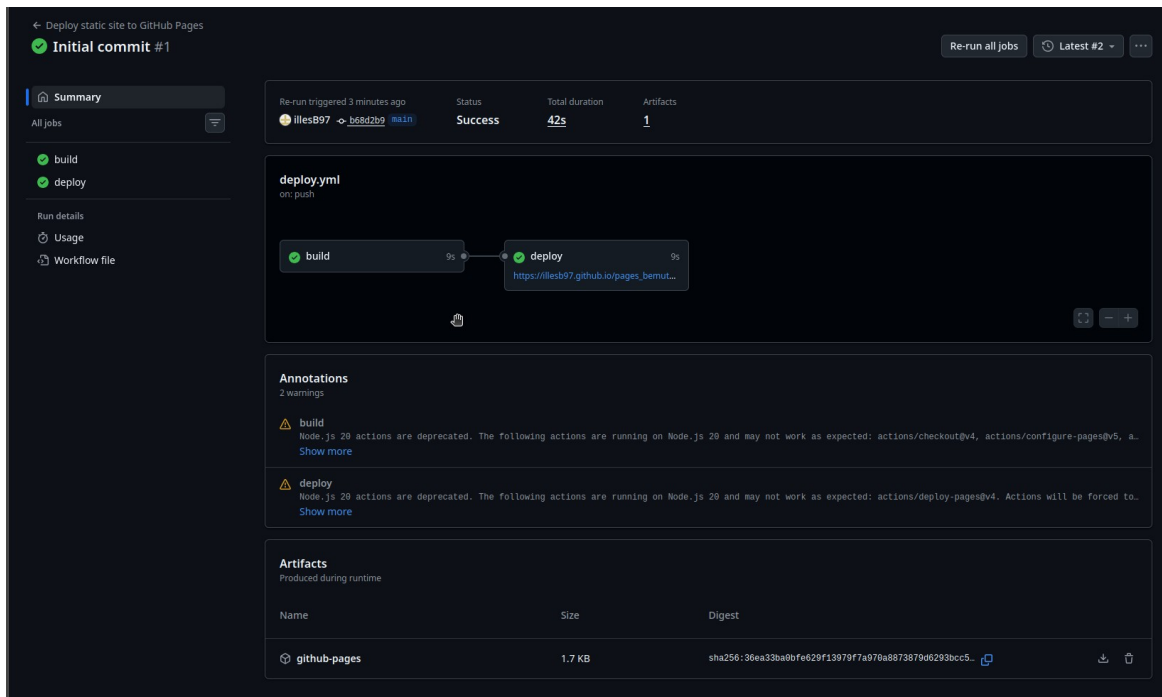
1. ábra. A `pages_bemutato` repository tartalma a GitHubon, a `main` ágon. A `.github/workflows` mappa a CI/CD workflow-fájlt, a `docs` mappa a publikálandó statikus tartalmat (HTML+CSS) tartalmazza.



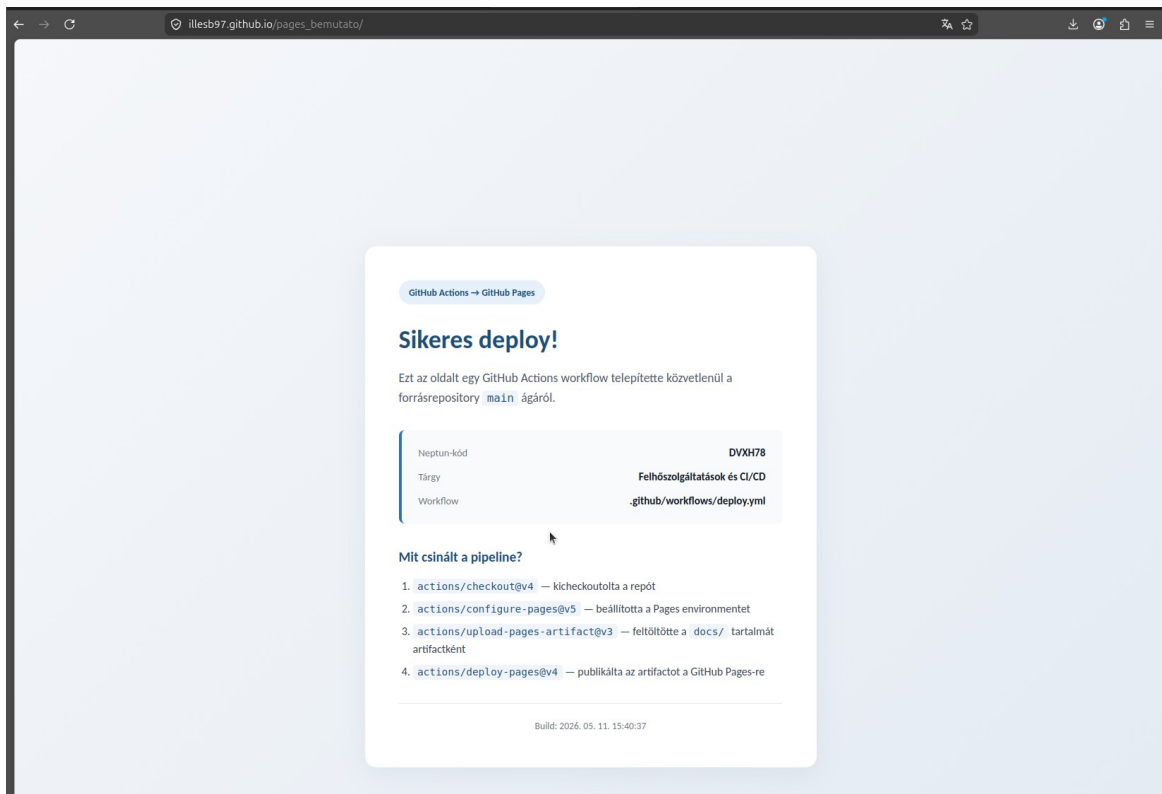
2. ábra. A `.github/workflows/deploy.yml` fájl tartalma a GitHub kódböngészőjében. Mind a négy lépés konkrét, verziózott action-t hív meg: `checkout@v4`, `configure-pages@v5`, `upload-pages-artifact@v3` és `deploy-pages@v4`.



3. ábra. A repository Settings → Pages menüpontja. A Source értéke GitHub Actions, a tetején a zöld sávval jelzett „Your site is live at https://illesb97.github.io/pages_bemutato/” sor, amely a publikált URL-t mutatja.



4. ábra. A workflow-futtatás részletes nézete az Actions fülön. A build és deploy job egymás után, mindkettő zöld pipával lezárult; a deploy doboz alatt látszik a publikált oldal URL-je. A teljes futási idő 42 másodperc, az artifact github-pages néven, 1,7 KB méretben jött létre.



5. ábra. A publikált weboldal a https://illesb97.github.io/pages_bemutato/ URL-en. A böngésző címsora is rajta van a képen, amely megerősíti, hogy a deploy valóban a GitHub Pages domainre került ki.

Bevált gyakorlatok

- **Action-verziók rögzítése.** Mindig konkrét főverzióra (pl. @v4) vagy commit-SHA-ra hivatkozzunk, ne @main-re. Így egy upstream törés nem rontja el a build-et észrevétlenül.
- **Legkisebb jogosultság.** A permissions blokkban csak azokat a hatásköröket engedélyezzük, amelyekre valóban szükség van; a többi alapértelmezetten none legyen.
- **Secret-kezelés.** API-kulcsokat, tokeneket sosem írunk a YAML-be. A GitHub repository, environment vagy szervezet szintű secretjeit használjuk, és ezeket a logokba kiírni tilos.
- **Cache.** A függőségek (npm, pip, NuGet stb.) cache-elésével – pl. actions/setup-node beépített cache opciójával – jelentősen csökkenthető a build ideje.
- **Status badge.** A README.md-be illesztett build-állapot jelvény (badge) azonnal mutatja, hogy a fő ág éppen zöld-e.
- **Branch protection.** Védett main ág és kötelező status check beállításával biztosítható, hogy csak sikeres workflow után lehessen merge-ölni.

Összefoglalás

Összefoglalva: a GitHub Actions a GitHub natív, YAML-alapú CI/CD megoldása, amely a repositoryval szorosan integrálódva nyújt automatikus fordítást, tesztelést és telepítést. A GitHub Pages-szel együtt egyetlen workflow-fájl elegendő ahhoz, hogy egy statikus weboldal verziókezelte forrásból, minden commit után, ember beavatkozása nélkül publikálódjon. A bemutatott példa minimális konfigurációval, a hivatalos action-ök legújabb verzióival mutatja be a teljes folyamatot, és kis kiegészítéssel tetszőleges statikus generátorra vagy Node.js-alapú frontend buildelési láncra kiterjeszthető.